

Arquitectura de computadoras

MUX 6:1 y operaciones instanciadas



CUCEI

Clave: IL365 NRC: 200204 Seccion: D03 Calendario: 2026B

Ariel Rojas Garcia

221930873

1. INTRODUCCIÓN

1.1 La ALU y las operaciones aritméticas

Esta actividad se centra en ese mecanismo de selección: implementar un multiplexor binario de 6 a 1 (Mux61) que, a partir de cuatro operandos (A, B, C, D) y una señal de selección (Sel), entrega el resultado de una de seis operaciones aritméticas posibles, junto con una señal adicional de habilitación (en) que indica si la operación seleccionada pertenece a un subconjunto marcado como prioritario.

1.2 Verilog y el estilo conductual

El módulo se describe en estilo conductual (behavioral), utilizando un bloque always combinacional sensible a todas sus entradas (always @(*)) y una sentencia condicional case sobre la señal Sel. En este estilo, toda señal asignada dentro de un bloque procedural debe declararse como tipo reg, a diferencia de las asignaciones continuas (assign), que solo pueden manejar tipo wire.

1.3 Temas de investigación

Jim Keller (ingeniero)

James "Jim" Keller es un ingeniero de microprocesadores estadounidense, egresado en ingeniería eléctrica de Pennsylvania State University, reconocido por más de cuatro décadas de trabajo en el diseño de arquitecturas de cómputo. Su aportación más citada a la industria de los procesadores es haber sido el arquitecto principal de la microarquitectura AMD K8 (base del Athlon 64), además de haber participado en el diseño del Athlon (K7), en la definición del conjunto de instrucciones x86-64 y en la interconexión HyperTransport. Posteriormente, tras la adquisición de P.A. Semi por parte de Apple, lideró el desarrollo de los sistemas en chip A4 y A5 usados en el iPhone y el iPad. Ya de vuelta en AMD, dirigió el diseño de la microarquitectura Zen, y más adelante trabajó en Tesla en el hardware de Autopilot, y en Intel como vicepresidente senior del grupo de ingeniería de silicio. Actualmente es director ejecutivo de Tenstorrent, una empresa enfocada en procesadores para inteligencia artificial.

¿Qué es una GPU?

Una GPU (Graphics Processing Unit) es un procesador especializado, originalmente diseñado para acelerar el renderizado de gráficos, que se ha extendido al cómputo paralelo de propósito general. A diferencia de una CPU, que cuenta con pocos núcleos muy complejos optimizados para baja latencia y control de flujo, una GPU integra miles de núcleos aritméticos simples agrupados en bloques (llamados streaming multiprocessors en la terminología de NVIDIA, o

compute units en la de AMD). Cada bloque comparte una unidad de control, un banco de registros y una memoria caché/compartida de acceso rápido, mientras que todos los bloques acceden a una memoria global de alto ancho de banda (VRAM). El modelo de ejecución típico es SIMT (Single Instruction, Multiple Threads): una misma instrucción se ejecuta simultáneamente sobre muchos datos distintos.

Una GPU conviene usarla cuando la tarea es masivamente paralela y aplica la misma operación sobre grandes volúmenes de datos independientes entre sí, por ejemplo: entrenamiento e inferencia de redes neuronales, procesamiento de imágenes y video, operaciones matriciales, simulaciones físicas y minería de criptomonedas. En cambio, una CPU sigue siendo más adecuada para tareas secuenciales con mucha lógica de decisión y dependencias entre pasos, donde la baja latencia por hilo importa más que el volumen total de operaciones paralelas.

ALU vs. GPU (parte aritmética)

La ALU de una CPU es una unidad única (o unas pocas, en diseños superescalares) capaz de ejecutar operaciones aritméticas y lógicas complejas con baja latencia, procesando instrucciones en gran medida de forma secuencial. La parte aritmética de una GPU, en cambio, puede entenderse como un arreglo de miles de ALUs simplificadas (núcleos CUDA, en el caso de NVIDIA) que ejecutan la misma operación aritmética sobre muchos datos a la vez. Es decir, la diferencia no es tanto qué operación se calcula, sino la escala y el modelo de ejecución: una ALU de CPU prioriza la latencia de una sola operación compleja, mientras que el conjunto de ALUs de una GPU prioriza el rendimiento agregado (throughput) sobre grandes volúmenes de operaciones simples y paralelas — el mismo tipo de arreglo de operaciones que se explora, a escala mucho menor, en el multiplexor de esta actividad.

CISC

CISC (Complex Instruction Set Computer) es un enfoque de arquitectura de conjunto de instrucciones en el que el procesador incluye instrucciones complejas, capaces de realizar en una sola instrucción tareas que equivaldrían a varios pasos más simples (por ejemplo, leer de memoria, operar y volver a escribir en la misma instrucción). Esto se logra mediante microcódigo interno, modos de direccionamiento variados e instrucciones de longitud variable — los "elementos complejos" o módulos adicionales de arquitectura que menciona la actividad. El ejemplo más conocido es la familia x86. Se contrapone al enfoque RISC (Reduced Instruction Set Computer), que usa instrucciones más simples y de duración fija,

delegando la composición de tareas complejas al software o al compilador.

2. OBJETIVO

Objetivo general

Diseñar, describir en Verilog y verificar mediante simulación un módulo Mux61 que implemente un multiplexor binario de 6 a 1 de forma conductual, integrando seis operaciones aritméticas sobre los operandos A (4 bits), B (4 bits), C (4 bits) y D (8 bits), con salidas R (8 bits) y en (1 bit).

Objetivos particulares

- Determinar el ancho de la señal de selección Sel necesario para direccionar 6 entradas.
- Implementar de forma independiente cada una de las seis operaciones (op1.v – op6.v) como módulos separados, instanciados dentro del módulo superior Mux61.
- Describir el bloque selector mediante la sentencia case dentro de un always combinacional.
- Generar la señal en, activa únicamente cuando la operación seleccionada esté marcada con ★ en la especificación (A+B–C, corrimiento de D, B+C).
- Garantizar que una selección inválida (Sel = 000 o 111) produzca R = 0 y en = 0.
- Construir un testbench que ejercite cada una de las seis operaciones cinco veces, con operandos variables, y validar los resultados contra el cálculo esperado.

3. DESARROLLO

Conforme a los entregables solicitados, esta práctica produce ocho archivos .v (op1.v–op6.v, Mux61.v y Mux61_tb.v) además de este reporte en PDF. A continuación se describe cada parte del diseño implementado, junto con las capturas del resultado obtenido en simulación.

3.1 Especificación de las seis operaciones

La Figura 2 de la actividad específica seis operaciones, tres de ellas marcadas con ★ (las que deben activar la salida en). A, B y C son de 4 bits; D es de 8 bits; R, la salida del multiplexor, es de 8 bits. Con 6 operaciones a seleccionar se necesitan como mínimo 3 bits en Sel ($2^3 = 8$ combinaciones posibles, de las cuales 6 son válidas y 2 quedan como casos de selección inexistente).

Sel	Módulo	Operación	★ (en)	Observación de ancho
001	op1.v	$A + B - C$	Sí	Se extiende a 8 bits antes de restar, para evitar wrap-around si $C > A+B$
010	op2.v	$A * C$	No	Máx. $15 \times 15 = 225$, cabe en 8 bits sin truncar
011	op3.v	$D \ll 2$ (corrimiento izq.)	Sí	Los bits que superan el bit 7 se truncan; es el comportamiento esperado
100	op4.v	$C * D$	No	Máx. $15 \times 255 = 3825$, no cabe en 8 bits: el resultado se trunca a los 8 bits menos significativos
101	op5.v	$D \bmod 2$	No	Equivale al bit menos significativo de D
110	op6.v	$B + C$	Sí	Máx. $15+15 = 30$, cabe en 8 bits sin truncar

Tabla 1. Especificación de las seis operaciones del Mux6_1

3.2 Módulos de operación (op1.v – op6.v)

Cada operación se implementó como un módulo combinacional independiente, con una sola asignación continua (assign), ya que ninguna requiere lógica secuencial.

3.3 Módulo superior Mux61 (Mux6_1.v)

El módulo superior instancia las seis operaciones, conectando A, B, C y D según lo que cada una necesita, y usa un bloque always combinacional con case sobre Sel para elegir cuál resultado enviar a R, además de fijar en según si la operación está marcada con ★.

Las salidas del multiplexor se conectan a las instancias mediante señales intermedias de tipo wire (r1 a r6), ya que la salida de un módulo instanciado no puede conectarse directamente a un puerto de entrada ni a una señal reg. El case incluye un default explícito para cubrir Sel = 000 y Sel = 111, forzando R = 0 y en = 0 según lo pide la especificación.

3.4 Operaciones a ejecutar y resultados esperados del testbench

Antes de construir el testbench, se definió qué operaciones se iban a ejercitar y qué resultado debía producir cada una, para poder contrastar ese cálculo manual contra la forma de onda real obtenida en ModelSim. El testbench recorrerá las seis operaciones (Sel = 001 a 110) cinco veces cada una, variando A, B, C y D en cada repetición, y además probará una vez los dos valores de Sel que no corresponden a ninguna operación (000 y 111), donde se espera R = 0 y en = 0.

A modo de ejemplo de verificación manual, para un primer conjunto de operandos fijo (A=5, B=2, C=3, D=20)

3.5 Testbench (Mux6_1_TB.v)

El testbench recorre las seis operaciones dentro de un ciclo for que se repite 5 veces, variando A, B, C y D en cada repetición mediante \$random, de manera que cada operación se ejerce cinco veces con operandos distintos.

3.6 Resultados y validación en ModelSim

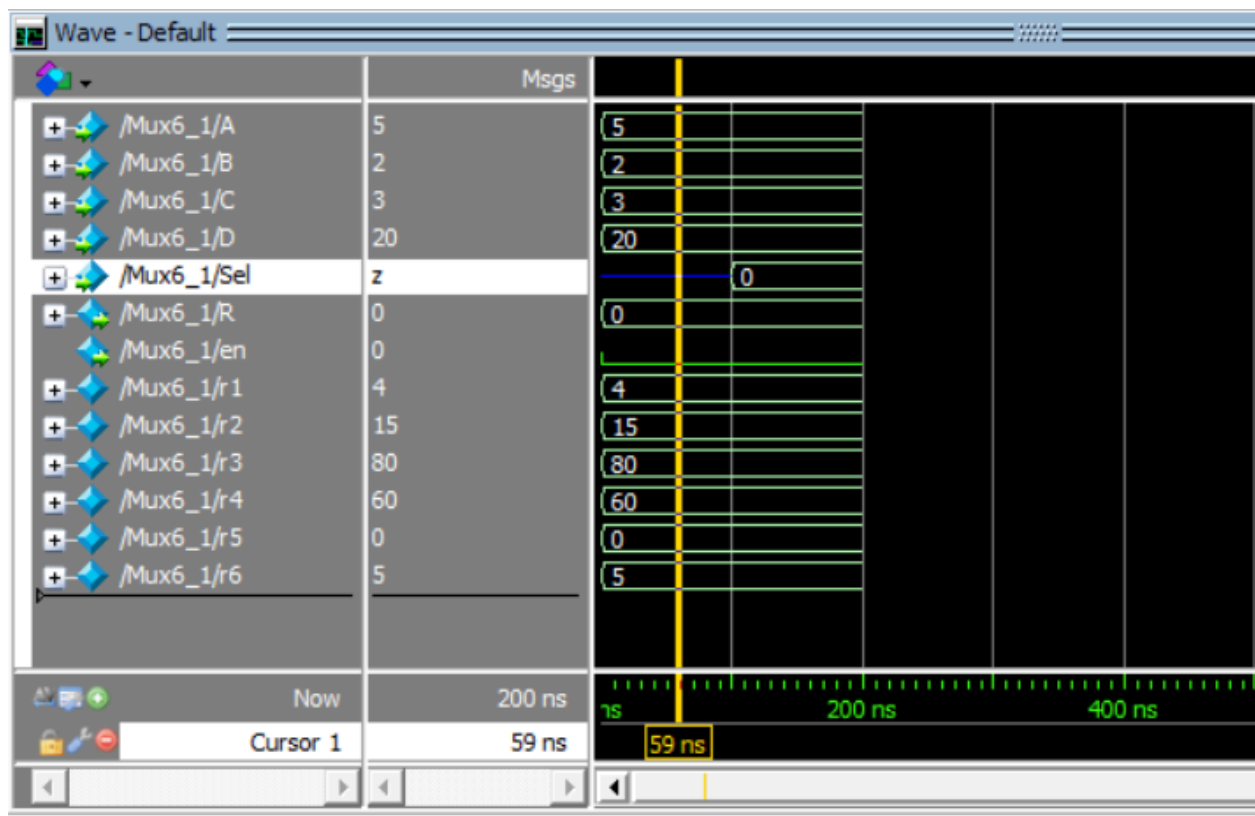


Figura 1. Ventana Wave de resultados obtenidos en la simulación Mux6_1.v

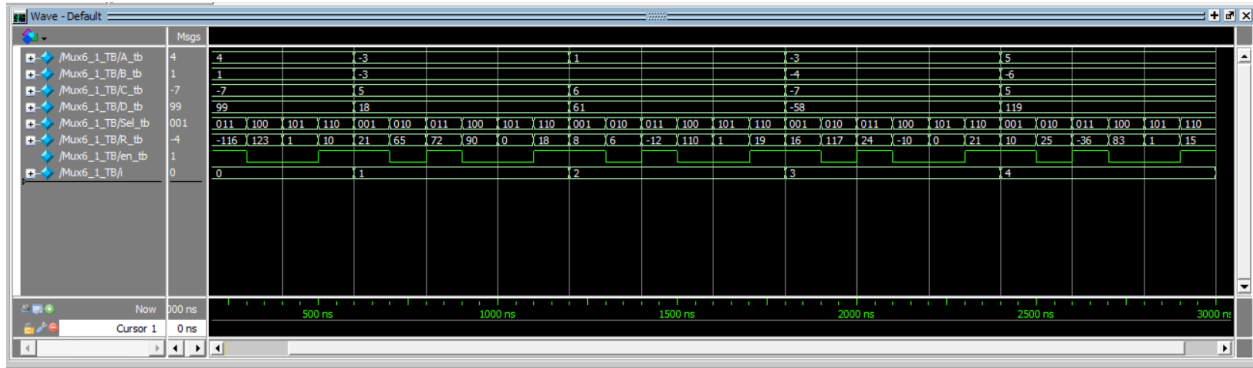


Figura 2. Ventana Wave de resultados obtenidos en la simulación Mux6_1_TB.v

4. CONCLUSIÓN

Esta actividad me permitió entender cómo un multiplexor puede organizar varias operaciones independientes bajo una sola señal de selección, en lugar de escribir todo el circuito como un bloque monolítico. Dividir cada operación en su propio módulo (op1.v–op6.v) y conectarlas después dentro de Mux6_1.v me ayudó a ver con más claridad la diferencia entre describir un circuito de forma estructural (instanciando módulos) y de forma conductual (el bloque always con case).

Durante el desarrollo cometí varios errores típicos de quien apenas se está acostumbrando a Verilog: dejé puertos de un módulo instanciado sin conectar, lo cual hacía que la señal correspondiente quedara en estado x en la simulación en lugar de fallar de forma evidente; olvidé encerrar en begin...end las asignaciones múltiples dentro de una misma rama del case, lo que provocaba que una de las asignaciones se ejecutara siempre, sin importar el valor de Sel; y en una primera versión invertí por error los valores de en para dos de las operaciones, asignando el habilitador a la operación equivocada. Corregir estos errores me ayudó a entender que en Verilog un error de sintaxis no siempre impide compilar — muchas veces el circuito compila pero simplemente no se comporta como se espera, y solo se detecta revisando la forma de onda con cuidado.

5. REFERENCIAS

- Mano, M. M., & Ciletti, M. D. Diseño Digital. Pearson Educación.
- Palnitkar, S. Verilog HDL: A Guide to Digital Design and Synthesis. Prentice Hall.
- Ciletti, M. D. Advanced Digital Design with the Verilog HDL. Pearson.
- Tocci, R. J., Widmer, N. S., & Moss, G. L. Sistemas Digitales: Principios y Aplicaciones. Pearson Educación.
- IEEE. IEEE Standard for Verilog Hardware Description Language (IEEE Std 1364). IEEE Computer Society.

- Hennessy, J. L., & Patterson, D. A. Computer Architecture: A Quantitative Approach. Morgan Kaufmann.
- Wikipedia contributors. Jim Keller (engineer). Wikipedia, The Free Encyclopedia. • AMD. Computer Architect Jim Keller Joins AMD as Chief of Processor Group (comunicado de prensa).
- NVIDIA Corporation. CUDA C++ Programming Guide — arquitectura de streaming multiprocessors.